ABSTRACT

Modern agriculture is steadily moving toward smarter and more efficient methods of growing crops. Greenhouses, in particular, require constant monitoring of temperature, humidity and soil conditions to ensure healthy plant growth. In many cases, farmers still depend on manual observation, which is time- consuming and often inaccurate.

To address this problem, this project introduces a **Real-Time IoT Enabled Precision Greenhouse Agriculture** System using the ESP32 microcontroller. The system continuously measures key environmental parameters such as temperature, humidity, soil moisture density and soil sensor voltage using low-cost sensors. The collected data is displayed instantly on an LCD screen for local monitoring and is also uploaded to the ThingSpeak cloud platform, allowing users to track greenhouse conditions from anywhere using a mobile or computer.

A relay-controlled water pump is included to demonstrate how irrigation can be automated or manually controlled in future versions. The system is designed to be simple, affordable and suitable for small greenhouses, nurseries, home gardens and academic IoT applications.

Overall, this project shows how IoT technology can make agriculture more precise, efficient and easier to manage by providing real-time insight into essential growing conditions.

TABLE OF CONTENTS

ACKNOWLEDGEMENT

ABSTRACT

LIST OF FIGURES

CHAPTER 1: INTRODUCTION **1-2**
1.1 Problem Statement 2
1.2 Objectives 2

CHAPTER 2: LITERATURE SURVEY **3-5**

CHAPTER 3: METHODOLOGY **6-10**
3.1 System Overview 6
3.2 Working Principle 6-7
3.3 Block Diagram 7-8
3.4 Flow Chart 8-9
3.5 Algorithm Description 9-10
3.5.1 Algorithm Steps
3.5.2 Flow Representation

CHAPTER 4: HARDWARE DESCRIPTION **11-18**
4.1 ESP32 Microcontroller 11-12
4.2 DHT11 Sensor 13
4.3 Soil Moisture Sensor 14
4.4 Relay Module 15
4.5 Water Pump 16
4.6 LCD Display 17
4.7 Power supply 18
4.8 Bread Board 18

CHAPTER 5: SOFTWARE DESCRIPTION **19-22**
5.1 Arduino IDE 19
5.2 ESP32 Board Package 19
5.3 Required Libraries 20
5.4 Program Flow 20-21
5.5 ThingSpeak Cloud Communication 21
5.6 LCD Display Programming 21
5.7 Relay Control Software 22
5.8 Testing and Debugging 22
5.9 Summary 22

CHAPTER 6: ADVANTAGES AND DISADVANTAGES **23-24**

6.1 Advantages 23

6.2. Disadvantages 24

CHAPTER 7: RESULT AND FUTURE ENHACEMENT **25-29**

REFERENCES

APPENDIX

LIST OF FIGURES

FIGURE No.	DESCRIPTION	PAGE No.
Fig 3.1	Block Diagram	7
Fig 3.2	Data flow chart	8
Fig 4.1	ESP32 Dev Module V1(Pin Diagram)	12
Fig 4.2	DHT11 Sensor	13
Fig 4.3	Soil Moisture Sensor	14
Fig 4.4	Relay Module	15
Fig 4.5	Mini DC water pump	16
Fig 4.6	LCD display(16x2)	17
Fig 7.1	Model set up	25
Fig 7.2	Working Model	26
Fig 7.3	LCD display	26
Fig 7.4	ThinkSpeak data visualization	27

CHAPTER 1:
INTRODUCTION

Agriculture is rapidly adopting digital technologies to improve crop productivity, reduce manual labor, and enable data-driven decision-making. Traditional greenhouse monitoring relies on manual observation of temperature, humidity, and soil moisture levels, which can be inaccurate and time-consuming. These limitations often lead to improper watering, poor environmental control, and reduced crop health.

To address these issues, this project presents a **Real-Time IoT Enabled Precision Greenhouse Agriculture**, designed to continuously measure key environmental parameters and upload them to the cloud in real time using **ThingSpeak IoT platform**. The system uses a **DHT11 sensor** to monitor temperature and humidity, and a **soil moisture sensor** to measure soil density and analog voltage of the soil condition. The sensed values are displayed locally on an **LCD module** and simultaneously uploaded to ThingSpeak for remote monitoring. A **relay-controlled water pump** is included for demonstration purposes, representing how irrigation can be automated or manually controlled in a greenhouse environment. The ESP32's inbuilt Wi-Fi capability ensures reliable data transmission without external hardware. This system enables farmers and researchers to track environmental conditions anytime, analyze trends using ThingSpeak graphs, and improve crop management and water conservation. Thus, the proposed system offers a low-cost, energy-efficient, and real-time solution for intelligent greenhouse monitoring.

1.1 Problem Statement

Conventional greenhouse setups rely heavily on manual observations to analyze environmental conditions like temperature, humidity, and soil moisture. These methods lack real-time insight, are prone to human error, and do not provide remote accessibility. As a result, plants may suffer from overwatering, underwatering, or exposure to unsuitable climatic conditions.

Although modern IoT technologies allow live monitoring, many farmers struggle with expensive systems or complex installations that require professional maintenance. There is a need for a simple, affordable, and IoT-enabled monitoring system that can collect data, display it locally, and upload it to the cloud for remote access.

The problem addressed in this project is the development of a low-cost IoT greenhouse monitoring system that accurately senses temperature, humidity, soil moisture density, and sensor voltage, and sends data to ThingSpeak for online visualization. The system must integrate sensors, a microcontroller, a cloud platform, and a demonstration pump into one reliable platform while ensuring easy deployment and scalability**.**

1.2 Objectives

i. To design and develop an IoT-based greenhouse monitoring system using ESP32 with Wi-Fi capability.

ii. To measure temperature and humidity using the DHT11 sensor and soil moisture density and voltage.

iii. To display real-time data on a 16×2 LCD module for local monitoring.

iv. To upload sensor data to **ThingSpeak cloud platform** for online visualization and remote monitoring.

v. To demonstrate irrigation control using a relay and DC water pump (manual or automated expansion possible).

vi. To create a cost-effective, scalable system suitable for small farms, research greenhouses, and academic applications

CHAPTER 2:

LITERATURE SURVEY

This literature survey analyses pervious research papers focused on smart greenhouse monitoring systems, wireless sensor-based agricultural applications, cloud-based farm management and irrigation automation methods. The reviewed studies explore diverse system designs, sensing mechanisms, data transmission techniques and control algorithms focused on the enhancing precision agriculture. These works provide a technical reference for our project **Real-Time IoT Enabled Precision Greenhouse Agriculture** and help identify research gaps related to system cost, ease of deployment, real-time local visualization, and scalability.

M. A. Zamora-Izquierdo et al., [1] Proposed **Smart Farming IoT Platform Based on Edge and Cloud.**

This work explains about a smart farming IoT platform integrates edge computing and cloud computing to improve agricultural monitoring and decision-making. The system uses IoT sensors to collect environmental data such as soil moisture, temperature, and humidity. Edge devices process data locally to reduce latency and bandwidth usage, while cloud servers perform long-term storage and advanced analytics. The proposed architecture improves system scalability, reliability, and real-time responsiveness. The results show that combining edge and cloud computing enhances precision agriculture applications and supports efficient farm management.

L. Ruiz-Garcia et al., [2] Proposed, **A Review of Wireless Sensor Technologies and Applications in Agriculture**

This work explains about the review of wireless sensor technologies used in agricultural applications. It discusses different types of sensors, wireless communication protocols, and network architectures suitable for monitoring environmental parameters such as temperature, humidity, soil moisture, and crop conditions. The paper highlights the advantages of wireless sensor networks in improving productivity, reducing labor, and enabling real-time monitoring. Challenges related to power consumption, communication range, and deployment cost are also discussed. The review serves as a foundational reference for smart agriculture and greenhouse monitoring systems.

A. Jawad, et al., [3] Proposed **Energy-Efficient Wireless Sensor Networks for Precision Agriculture**

This work particularly focuses on the design of energy-efficient wireless sensor networks (WSNs) for precision agriculture applications. The proposed system aims to reduce power consumption while maintaining reliable data transmission from agricultural fields. Various routing strategies, communication protocols, and sensor deployment methods are analyzed. The study demonstrates that optimized energy management significantly extends network lifetime and improves monitoring performance. The proposed approach is suitable for long-term agricultural monitoring, including greenhouse and irrigation systems.

J. Gutierrez, et al., [4] **Proposed Automated Irrigation System Using a wireless sensor network for Precision Agriculture**

Their work explains about an automated irrigation system based on a wireless sensor network. The system uses soil moisture sensors to monitor water content in agricultural fields and automatically controls irrigation based on predefined threshold values. Sensor data is transmitted wirelessly to a central control unit, which activates irrigation when required. The experimental results show significant water savings and improved irrigation efficiency compared to traditional methods. The system demonstrates the effectiveness of IoT-based automation in agricultural water management.

S. Li, et al., [5] **Proposed the Internet of Things: A Survey**

Their work survey explains about an overview of the Internet of Things (IoT), including its architecture, enabling technologies, applications, and challenges. It discusses sensing technologies, communication protocols, data management, and security issues related to IoT systems. The paper highlights the role of IoT in domains such as smart cities, healthcare, and agriculture. It also identifies future research directions and open challenges. This survey serves as a theoretical foundation for IoT-based systems, including smart greenhouse monitoring applications.

B. George and L. Mathew [6] **Proposed Smart Greenhouse with Real-Time Monitoring Using ESP8266**

This work explains about a smart greenhouse monitoring system using the ESP8266 Wi-Fi module. The system monitors environmental parameters such as temperature, humidity, and soil moisture using low-cost sensors. Sensor data is transmitted to a cloud platform for real-time visualization and remote monitoring. The proposed system provides a cost-effective and user-friendly solution suitable for small-scale greenhouses. Experimental results demonstrate reliable data transmission and effective monitoring, making the system useful for precision agriculture and IoT- based farming applications.

P. K. Pandit, et al., [7] **Proposed IoT Based Green House Monitoring and Controlling System**

Their work explains about an IoT-based greenhouse monitoring and controlling system designed to improve crop growth by continuously observing environmental conditions. The system uses sensors to measure parameters such as temperature, humidity, and soil moisture. A microcontroller processes the sensor data and transmits it through Wi-Fi for remote monitoring. Based on predefined threshold values, a relay-controlled water pump is automatically activated to irrigate plants when soil moisture falls below the required level. Additional components such as light and rain sensors are used to enhance greenhouse automation. The proposed system reduces manual effort, improves water management, and provides an efficient solution for smart agriculture applications. However, the system focuses mainly on automation and does not include advanced cloud-based data analytics or local visualization features.

CHAPTER 3:

METHODOLOGY

This chapter explains the overall methodology adopted for designing and implementing the **Real-Time IoT Enabled Precision Greenhouse Agriculture**. The system integrates multiple sensors, a Wi-Fi enabled microcontroller, a cloud platform, and an output module to continuously monitor environmental conditions and visualize them remotely using ThingSpeak.

3.1 System Overview

The proposed system continuously collects greenhouse environmental data using sensors such as **DHT11** (temperature & humidity) and a **soil moisture sensor** (soil density and voltage level). These values are processed by the **ESP32 Dev Module V1**, which acts as the central controller of the system. The processed data is then:

1. Displayed on a **16×2 LCD module** for local monitoring.
2. Uploaded to **ThingSpeak IoT cloud platform** using Wi-Fi for remote visualization.
3. Used to demonstrate irrigation control through a **relay-operated water pump**.

This combination of hardware and cloud integration ensures that users can monitor greenhouse conditions both locally and remotely in real-time.

3.2 Working Principle

The working of the system can be divided into four major stages:

1. Sensing Stage

- The **DHT11 sensor** measures the temperature and humidity of the surrounding greenhouse environment.
- The **soil moisture sensor** detects the moisture density of the soil using analog values.
- The sensor also provides **voltage output**, indicating dryness or wetness level.

2. Processing Stage

- The ESP32 reads sensor values through its **GPIO and ADC pins**.
- Data is converted into meaningful values:
    - Temperature (°C)
    - Humidity (%)
    - Soil moisture density (ADC value or percentage)
    - Voltage level (V)

  The microcontroller prepares this data for display and transmission.

3. **Display Stage**
   - A **16×2 LCD** connected via the I2C interface displays real-time values.
   - This ensures local monitoring without needing internet access.
4. **Cloud Upload Stage**
   - Using the ESP32's built-in Wi-Fi module, data is uploaded to **ThingSpeak Channel Fields**.
   - Fields used:
     - Field 1 → Temperature
     - Field 2 → Humidity
     - Field 3 → Soil Moisture Density
     - Field 4 → Sensor Voltage
   - ThingSpeak graphs help visualize historical and real-time trends.
5. **Actuation Stage (Demonstration)**
   - A **relay module** is connected to the ESP32.
   - The relay controls the **water pump**.
   - In this version of the project, the pump is used only for demonstration and for automatic irrigation.

3.3 Block Diagram

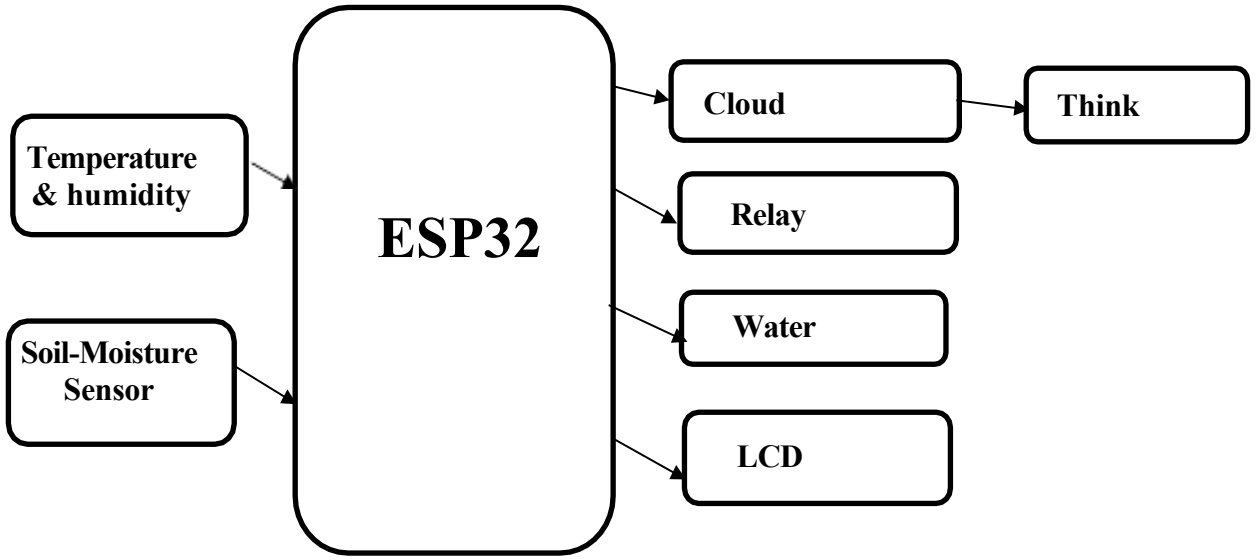


Fig 3.1 Block diagram

The figure 3.1, represents the complete working of the Real-Time IoT Enabled Precision Greenhouse Agriculture system. At the center of the model is the ESP32 microcontroller, which coordinates all sensing, processing, and communication tasks. The DHT11 sensor is connected to the ESP32 to measure the greenhouse's temperature and humidity and send the readings digitally. The soil moisture sensor is connected through an analog input pin and provides both moisture level and voltage output, which the ESP32 converts into readable values. All the collected data is displayed instantly on a 16×2 I2C LCD, allowing local monitoring even without internet connectivity. The ESP32 also uses its built-in Wi-Fi to upload all sensor readings to the ThingSpeak cloud platform, where the data is stored and plotted as graphs for remote monitoring. A relay module is connected to the ESP32 to control a small water pump, demonstrating how irrigation can be automated. Overall, the block diagram clearly shows how the system performs sensing, processing, displaying, uploading, and pump control in a single integrated IoT-based greenhouse monitoring setup.

3.4 Flow Chart

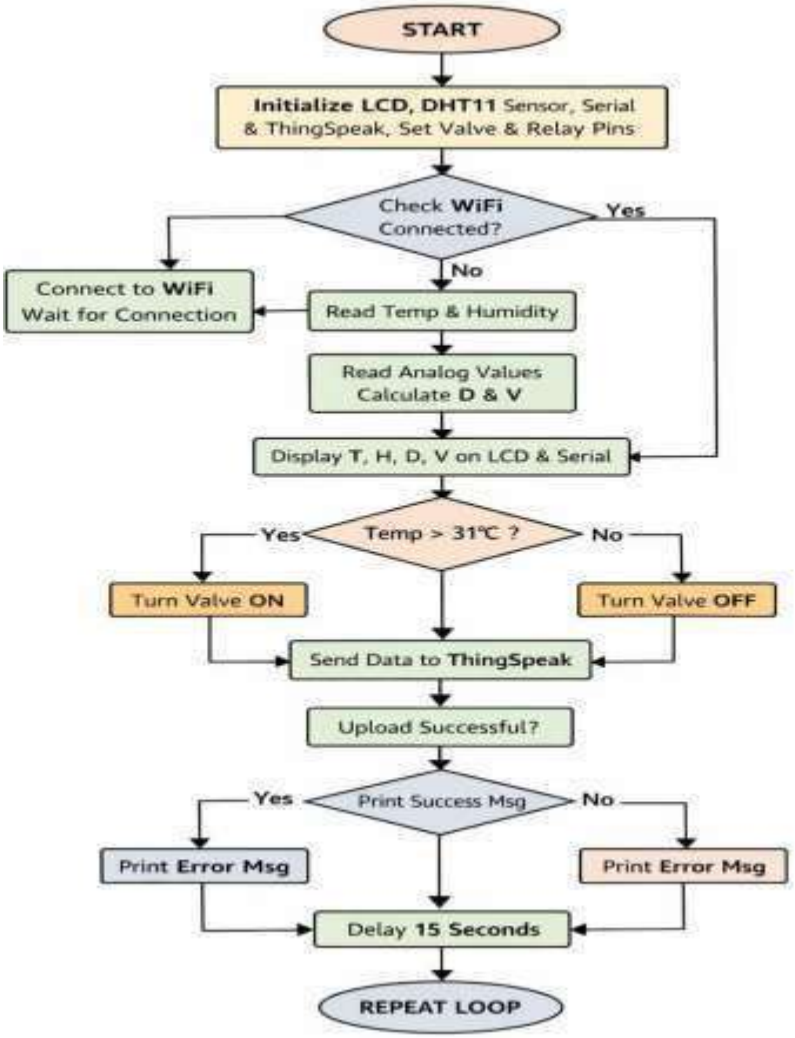


Fig 3.2. Data flow chart

- ESP32 Microcontroller:
  Acts as the main control unit. It reads sensor data, processes values, controls the valve, updates LCD, and sends data to ThingSpeak using WiFi.
- DHT11 Sensor:
  Measures temperature and humidity and sends digital data to the ESP32.
- Analog Sensors (Pins 34 & 35):
  Provide analog input values which are converted into D value and voltage using ESP32 ADC.
- Decision Logic:
  If temperature exceeds 31°C, the ESP32 turns ON the valve, otherwise it turns OFF.
- Relay / Valve:
  Controlled through ESP32 GPIO pins for automated control.
- LCD Display (16×2):
  Displays real-time values of temperature, humidity, D value, and voltage.
- WiFi Module (ESP32 has built-in):
  Connects ESP32 to the internet for cloud communication.
- ThingSpeak Cloud:
  Stores and visualizes sensor data for monitoring and analysis.

3.5 Algorithm Description

The Real-Time IoT Enabled Precision Greenhouse Agriculture system operates based on a real-time sensor data acquisition and cloud communication algorithm implemented using the ESP32 microcontroller. The algorithm begins by initializing the ESP32 along with all connected sensors, including the DHT11 temperature and humidity sensor and the soil moisture sensor. Output devices such as the 16×2 LCD display and the relay module are also initialized at the start of the process.

After initialization, the ESP32 establishes a Wi-Fi connection to enable communication with the ThingSpeak cloud platform. Once connected, the system continuously monitors environmental parameters such as temperature, humidity, soil moisture density, and sensor voltage. The acquired sensor data is processed and displayed locally on the LCD module, allowing on-site monitoring without dependency on internet connectivity.

Simultaneously, the processed data is transmitted to the ThingSpeak cloud platform for remote monitoring and visualization. A relay-controlled water pump is included in the system to demonstrate irrigation control. The algorithm executes continuously in a loop with a fixed delay between successive readings to ensure stable operation and reliable cloud updates.

3.5.1 Algorithm Steps

1. Start the system
2. Initialize ESP32 microcontroller
3. Initialize DHT11 sensor
4. Initialize soil moisture sensor
5. Initialize LCD display and relay module
6. Connect ESP32 to Wi-Fi network
7. Read temperature and humidity values
8. Read soil moisture density and sensor voltage
9. Display sensor values on LCD
10. Upload sensor data to ThingSpeak cloud
11. Control relay for water pump demonstration
12. Introduce a fixed delay
13. Repeat steps 7 to 12 continuously

3.5.2 Flow Representation

The algorithm follows a continuous loop-based execution model (refer figure 3.2) where sensor data acquisition, local display, cloud communication, and relay control are performed repeatedly. This ensures real-time monitoring of greenhouse conditions with both local and remote accessibility.

CHAPTER 4:

HARDWARE DESCRIPTION

This chapter explains the hardware components used in the **Real-Time IoT Enabled Precision Greenhouse Agriculture**. Each component plays a specific role in sensing, processing, displaying, or transmitting environmental data. The integration of these modules enables accurate monitoring of greenhouse conditions and real-time data upload to ThingSpeak.

Hardware Requirements

- ESP32 Microcontroller
- DHT11 Sensor
- Soil-Moisture Sensor
- Relay Module
- Water Pump
- LCD Display (16x2)
- Power Supply
- Bread board

Components

4.1 ESP32 Microcontroller

The ESP32 Dev Module V1 (figure 4.1) is a powerful, low-cost development board that integrates a dual-core Wi-Fi and Bluetooth-enabled microcontroller. It is widely used in IoT, automation, greenhouse monitoring, wireless sensing, and embedded applications due to its fast processing speed, built-in ADC pins, and wide range of communication interfaces. The board also includes multiple digital and analog pins, making it ideal for sensor integration, cloud communication, and real-time data monitoring.

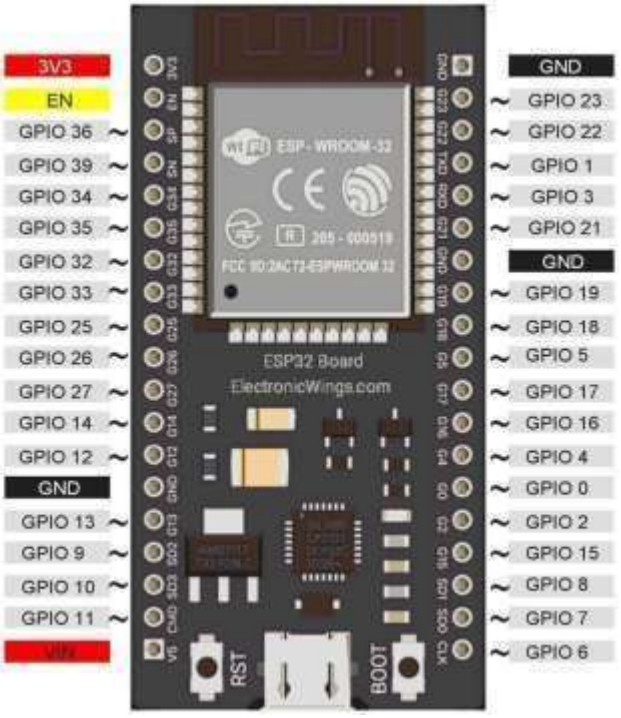


Fig 4.1. ESP32 Dev Module V1(Pin Diagram)

4.1.1 Specifications of ESP32

- Single or Dual-Core 32-bit LX6 Microprocessor with clock frequency up to 240 MHz.
- 520 KB of SRAM, 448 KB of ROM and 16 KB of RTC SRAM.
- Supports 802.11 b/g/n Wi-Fi connectivity with speeds up to 150 Mbps.
- Support for both Classic Bluetooth v4.2 and BLE specifications.
- 34 Programmable GPIOs.
- Up to 18 channels of 12-bit SAR ADC and 2 channels of 8-bit DAC
- Serial Connectivity include 4 x SPI, 2 x I^2C, 2 x I^2S, 3 x UART.
- Ethernet MAC for physical LAN Communication (requires external PHY).
- 1 Host controller for SD/SDIO/MMC and 1 Slave controller for SDIO/SPI.
- Motor PWM and up to 16-channels of LED PWM and secure boot & flash encryption.

4.2 DHT11 Sensor

As shown fig 4.2, The DHT11 is a digital sensor designed for measuring temperature and relative humidity. It combines sensing elements with a calibrated digital signal output and operates via a single data pin.

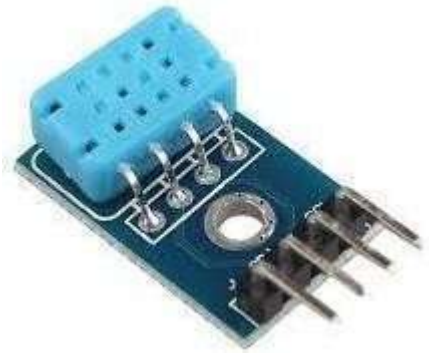


Fig 4.2. DHT11 sensor

4.2.1 Specifications

1. Measures both temperature and humidity
2. Operating voltage: 3.3V to 5V
3. Temperature range: 0°C to 50°C
4. Humidity range: 20% to 90% RH
5. Temperature accuracy: ±2°C
6. Humidity accuracy: ±5%
7. Digital single-wire output
8. Sampling rate: 1 reading per second
9. Low power consumption
10. 3-pin device (VCC, Data, GND)

4.2.2 Functions

- Measures greenhouse temperature
- Measures humidity levels
- Sends digital readings directly to ESP32
- Helps monitor plant growth environment

4.3 Soil-Moisture Sensor

As shown fig 4.3, The soil moisture sensor is used to measure soil density, dryness, and voltage output. It consists of sensing electrodes that detect electrical resistance or capacitance in the soil, depending on the model used problem statement.

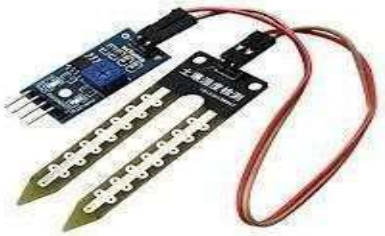


Fig 4.3. Soil-Moisture sensor

4.3.1 Specifications

1. Works on change in soil conductivity
2. Operating voltage: 3.3V to 5V
3. Analog output (AO) gives moisture value
4. Moisture detection range: 0% to 100%
5. Compatible with ESP32 ADC input
6. Low current consumption
7. Probe made for direct soil insertion
8. Output varies with water content
9. Easy to interface
10. Suitable for basic irrigation detection

4.3.3 Functions

- Detects soil dryness or wetness
- Provides analog value (density) to ESP32
- Provides raw voltage reading (V)
- Helps evaluate irrigation requirement

4.4 Relay Module

Relay module (Fig 4.4) acts as an electrically operated switch that allows the ESP32 to control high-current loads like the water pump.



Fig 4.4. Relay Module

4.4.1 Specifications

1. Coil operating voltage: 5V
2. Trigger level: 3.3V–5V compatible with ESP32
3. Switching capacity: 250V AC / 30V DC
4. Contacts available: COM, NO, NC
5. Opto-isolated versions available
6. Active-LOW or active-HIGH depending on module
7. Provides electrical isolation
8. Controls pumps, motors, and loads
9. Built-in flyback diode
10. Ideal for automation demonstration

4.4.2 Functions

- Controls the ON/OFF state of the water pump
- Provides electrical isolation between ESP32 and pump power
- Used only for demonstration (not automatic irrigation)

4.5 Water Pump

A DC water pump (Fig 4.5) is included to simulate irrigation in greenhouse systems. It is used to demonstrate how water may be circulated or supplied to plants.



Fig 4.5. Mini DC water pump

4.5.1 Specifications

1. Operating voltage: 5V DC
2. Current draw: 200 mA – 500 mA
3. Submersible type
4. Small plastic body, waterproof
5. Medium flow rate (80–120 L/h)
6. Works via relay control
7. Suitable for small irrigation demo
8. Low noise operation
9. Requires external 5V adapter
10. Cannot be powered from ESP32 pins

4.5.2 Functions

- Demonstration of irrigation mechanism
- Controlled manually through relay switching
- Helps simulate greenhouse watering concept

4.6 LCD Display (16x2)

The figure 4.6, module is used for visualizing live sensor data on the project board. It operates using either a parallel interface or an I2C interface



Fig 4.6. LCD display (16x2)

4.6.1 Specifications

1. 16 characters × 2 rows
2. Uses I2C communication (SDA, SCL)
3. Operating voltage: 5V
4. Low power backlit display
5. Built-in contrast control via potentiometer
6. I2C address: 0x27 or 0x3F
7. Requires only two data pins
8. Blue/green backlight
9. Works with LiquidCrystal_I2C library
10. Fast data update for real-time display

4.6.2 Functions

4.6.2.1 Displays temperature, humidity, soil moisture density, and voltage

4.6.2.2 Provides local monitoring without IoT access

4.6.2.3 Helps in testing and demonstration

4.7 Power Supply

The power supply is responsible for providing stable voltage to all components in the IoT-based greenhouse system. The ESP32 is powered through its USB port, which delivers 5V and internally converts it to 3.3V for its processor and Wi-Fi functions. The DHT11 sensor, LCD display, and relay module all require 5V and are powered directly from the ESP32's 5V pin. The soil moisture sensor is powered from the 3.3V pin since it outputs an analog signal compatible with the ESP32's ADC. The water pump, which draws higher current, is powered using an external 5V adapter rated at 1A–2A. The relay acts as a switch between the adapter and the pump, preventing high current from reaching the ESP32. All components share a common ground to ensure proper signal reference. A stable power supply ensures accurate measurements, safe switching, and reliable system operation.

4.8 Breadboard

A breadboard is a reusable prototyping board used to build electronic circuits without soldering. In this project, the breadboard plays an important role by allowing all the components—such as the ESP32, DHT11 sensor, soil moisture sensor module, LCD connections, and relay input wires—to be connected in an organized and safe manner. It provides internal metal strips that create automatic electrical connections when jumper wires and components are inserted into the holes. This makes it easy to test, modify, or replace parts during development. The breadboard helps in neatly distributing 5V, 3.3V, and GND lines from the ESP32 to different modules. It also allows the relay signal wire and sensor wires to be routed cleanly without loose connections. Using a breadboard reduces wiring errors, avoids short circuits, and makes debugging simpler. Overall, it acts as a stable and flexible platform for assembling the entire IoT greenhouse prototype before finalizing the circuit.

CHAPTER 5:

SOFTWARE DESCRIPTION

This chapter describes the software components used to develop the Real-Time IoT Enabled Precision Greenhouse Agriculture. The system is implemented using Arduino IDE, integrated libraries, and the ThingSpeak API. The ESP32 processes sensor data, displays values on the LCD, and uploads selected parameters to the cloud. The entire software workflow ensures accurate data acquisition, wireless transmission, and real-time visualization.

5.1 Arduino IDE

The Arduino Integrated Development Environment (IDE) is used to write, compile, and upload code to the ESP32 Dev Module V1. The IDE supports board configuration, library installation, and serial monitoring for debugging.

Features of Arduino IDE

- User-friendly interface
- Cross-platform support (Windows, Linux, Mac)
- Extensive library support
- Simple code compilation & uploading
- Compatible with ESP32 board package

Role in Project

- Writing the main program for ESP32
- Compiling sensor interfacing code
- Uploading final firmware to the microcontroller
- Monitoring sensor data using Serial Monitor

5.2 ESP32 Board Package

Since the ESP32 is not included in Arduino by default, the following Board Manager URL must be added**:** https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json

After adding the link, the “ESP32 Dev Module” board is installed. This enables full support for Wi-Fi, ADC, GPIOs, and built-in features.

5.3 Required Libraries

The following libraries are used in the program to interface sensors, display data, connect to Wi-Fi, and upload data to the cloud:

1. LiquidCrystal_I2C.h
    - Used to control the 16×2 LCD display through I2C communication.
    - Displays temperature, humidity, soil moisture density, and voltage locally.
2. DHT.h
    - Used to interface the DHT11 sensor.
    - Reads temperature and humidity values from the greenhouse environment.
3. WiFi.h
    - Provides Wi-Fi connectivity for the ESP32.
    - Used to connect the system to the internet for cloud communication.
4. ThingSpeak.h
    - Used to send sensor data to the ThingSpeak IoT cloud platform.
    - Enables remote monitoring and data visualization.

5.4 Program Flow

The program executes in two main parts: **setup ()** and **loop ()**.

1. Initialization Phase (setup function)

1. Initialize the LCD display and turn on backlight
2. Initialize the DHT11 temperature and humidity sensor
3. Start serial communication for debugging
4. Initialize ThingSpeak communication
5. Set valve and relay pins as output
6. Provide a small delay to stabilize hardware

2. Execution Phase (loop function)

1. Check Wi-Fi connection
    - If not connected, attempt to reconnect to the specified SSID
2. Read sensor values
    - Read temperature from DHT11
    - Read humidity from DHT11
    - Read analog values from soil moisture and voltage sensors
3. Process sensor data
    - Convert raw analog readings into meaningful values

4. Display data on LCD
    - Display temperature and humidity on the first row
    - Display density and voltage on the second row
5. Control logic
    - If temperature > 31°C → turn valve ON
    - If temperature < 31°C → turn valve OFF
6. Upload data to ThingSpeak
    - Temperature → Field 1
    - Humidity → Field 2
    - Density → Field 3
    - Voltage → Field 4
7. Wait for 15 seconds
8. Repeat the process continuously

5.5 ThingSpeak Cloud Communication

ThingSpeak is used as the IoT cloud platform for this project. Steps to communicate with ThingSpeak

- Create a ThingSpeak channel
    - Enable fields:

    Field 1: Temperature

    Field 2: Humidity

    Field 3: Soil Moisture Density

    Field 4: Sensor Voltage

    - Generate a Write API Key
    - Use ESP32 to send data using an HTTP GET request

Benefits

- Free real-time graph visualization
- Historical data storage
- Accessible from any device
- No external servers required

5.6 LCD Display Programming

The LCD is updated continuously with sensor values. Data formatting ensures that:

- First row displays **temperature & humidity**
- Second row displays **density & voltage**

5.7 Relay Control Software

The relay is used only for pump demonstration.

- A digital pin controls the relay
- HIGH or LOW signal activates or deactivates the pump
- Code includes relay initialization but does not automate irrigation unless required

5.8 Testing and Debugging

Testing is performed using:

- Serial Monitor for raw sensor values
- LCD display for real-time confirmation
- ThingSpeak graphs for cloud updates

Debugging steps include checking Wi-Fi connectivity, API key correctness, wiring connections, and ADC calibration.

5.9 Summary

The software implementation plays a crucial role in enabling real-time monitoring and control of the proposed IoT-based greenhouse system. The entire application is developed using the Arduino Integrated Development Environment (IDE) and deployed on an ESP32 microcontroller. The program is designed to handle multiple tasks, including sensor data acquisition, local display management, cloud communication, and control signal generation.

Sensor data such as temperature and humidity are obtained using the DHT11 sensor, while soil moisture density and voltage values are acquired through the ESP32's analog-to-digital converter(ADC). These parameters are processed and displayed on a 16×2 LCD, allowing users to observe greenhouse conditions directly at the site. To enable remote monitoring, the ESP32 establishes a Wi-Fi connection and uploads the sensor readings to the ThingSpeak cloud platform at regular intervals.

The software also includes a simple control mechanism in which a relay-operated valve is activated or deactivated based on a predefined temperature threshold. This demonstrates the system's capability for automated control. Continuous execution of the program is ensured using a loop-based structure with fixed delays, providing stable data updates and efficient operation.

Overall, the software architecture ensures accurate data collection, reliable wireless transmission, and real-time visualization, making the system effective for precision greenhouse agriculture and scalable for future enhancements.

CHAPTER 6:

ADVANTAGES AND DISADVANTAGES

6.1. Advantages

i. Real-time greenhouse monitoring:

The system continuously monitors temperature, humidity, soil moisture density, and voltage levels, allowing farmers to understand greenhouse conditions instantly.

ii. Reduced manual intervention:

Automation of data collection and valve control minimizes the need for constant human supervision inside the greenhouse.

iii. Efficient water management:

By monitoring environmental parameters, the system helps in preventing over-irrigation and promotes efficient water usage.

iv. Remote monitoring capability:

Sensor data is uploaded to the ThingSpeak cloud, enabling farmers to monitor greenhouse conditions from any location with internet access.

v. Low-cost implementation:

The use of ESP32 and basic sensors makes the system affordable and suitable for small and medium-scale farmers.

vi. Local data visualization:

The LCD display provides real-time on-site information, ensuring system usability even when internet connectivity is unavailable.

6.2. Disadvantages

i. Limited sensor accuracy:

Low-cost sensors such as DHT11 may not provide highly accurate readings compared to industrial-grade sensors.

ii. Dependence on Wi-Fi connectivity:

Cloud-based monitoring requires stable internet access, which may not be available in all rural areas.

iii. Basic automation logic:

The control mechanism is based only on temperature thresholds and does not consider multiple environmental factors.

iv. Scalability: limitations

The system may require additional hardware and configuration to support large-scale greenhouse operations.

v. No advanced data analytics:

The system does not include predictive analysis or intelligent decision-making based on historical data.

vi. Limited security features:

Data transmission to the cloud lacks encryption and advanced security mechanisms.

CHAPTER 7:

RESULTS AND FUTURE ENHACEMENT

The performance of the Real-Time IoT Enabled Precision Greenhouse Agriculture System was tested in both indoor and outdoor settings to evaluate its effectiveness in practical conditions. As shown in figure 7.2, the working model of the ESP32 microcontroller as successfully managed all connected sensors and components, including the DHT11 temperature and humidity sensor, soil moisture sensor, LCD display, relay module, and water pump, it is ensuring the continuous data collection, stable Wi-Fi communication, and reliable visualization both locally and on the cloud. The DHT11 sensor accurately capture changes in temperature and humidity according to the surrounding environment, while soil moisture readings effectively differentiated between wet and dry conditions, with sensor voltage measurements remaining within expected ranges to confirm proper operation and monitor soil conductivity. The relay module responded correctly to ESP32 control signals, activating and deactivating the water pump as needed, demonstrating the system's capability for automated irrigation management. Collectively, these observations indicate that the system operates reliably, delivering precise sensing, consistent data processing, clear local display, and dependable control functions, highlighting its suitability for smart greenhouse monitoring and precision agriculture applications.

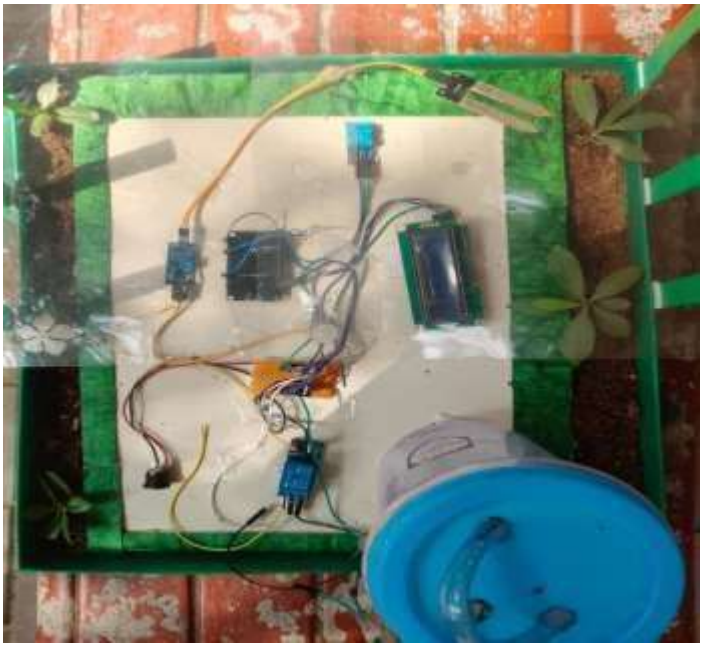


Fig 7.1 Model set up

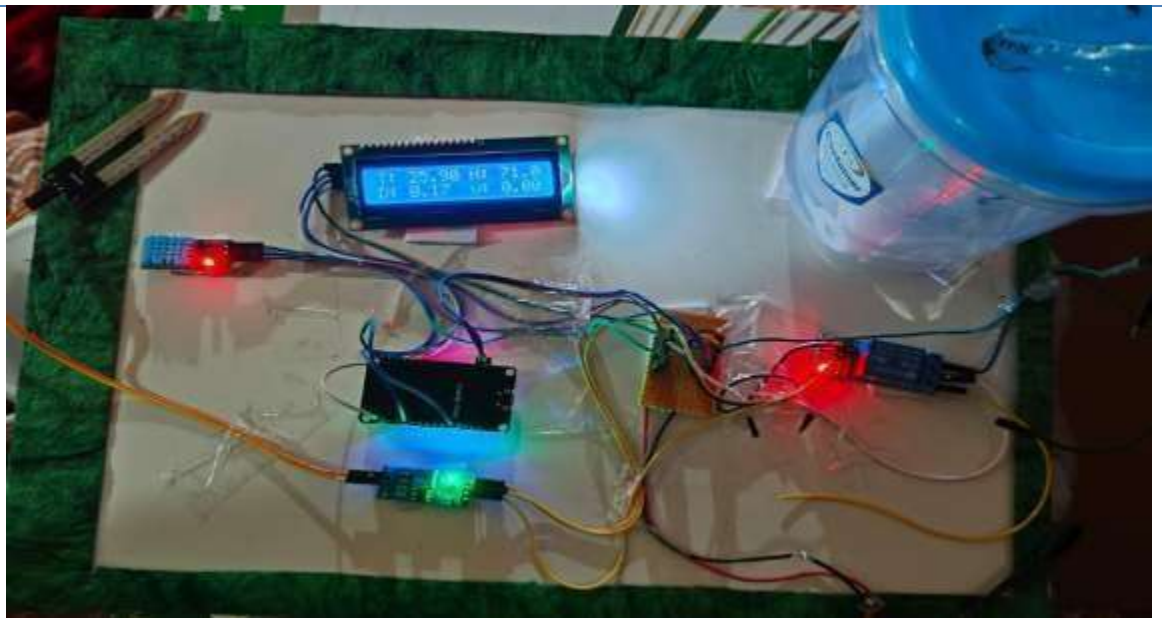


Fig 7.2 Working Model

The Figure 7.3 gives 16×2 LCD display and offers a clear and immediate display of sensor data, showing temperature and humidity on the first row, and soil moisture levels along with voltage on the second row, allowing users to access important information even without an active internet connection. At the same time, all sensor readings were transmitted to the ThingSpeak cloud platform every 15 seconds, with separate fields for temperature(T), humidity(H), soil moisture(D), and voltage(V). Figure 7.4 shows the screenshots of the cloud platform generated time-stamped logs and real-time graphs, enabling remote monitoring through smartphones. This integration of local and cloud-based visualization provided an effective solution for monitoring greenhouse conditions and analyzing environmental trends over time.

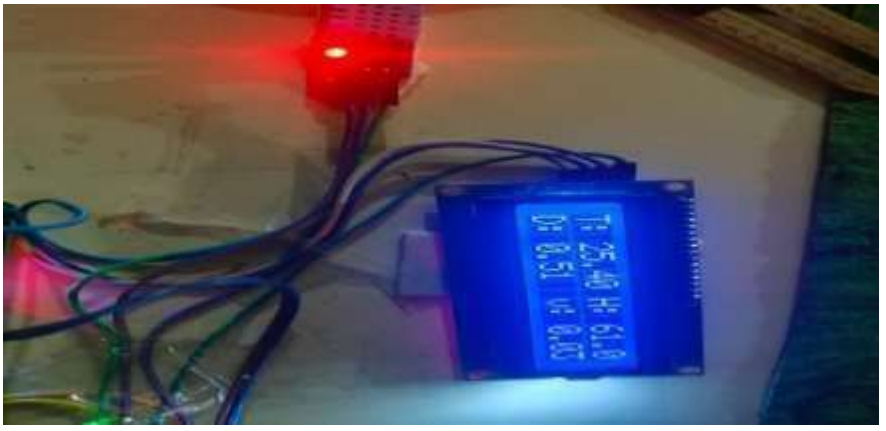


Fig 7.3 LCD display

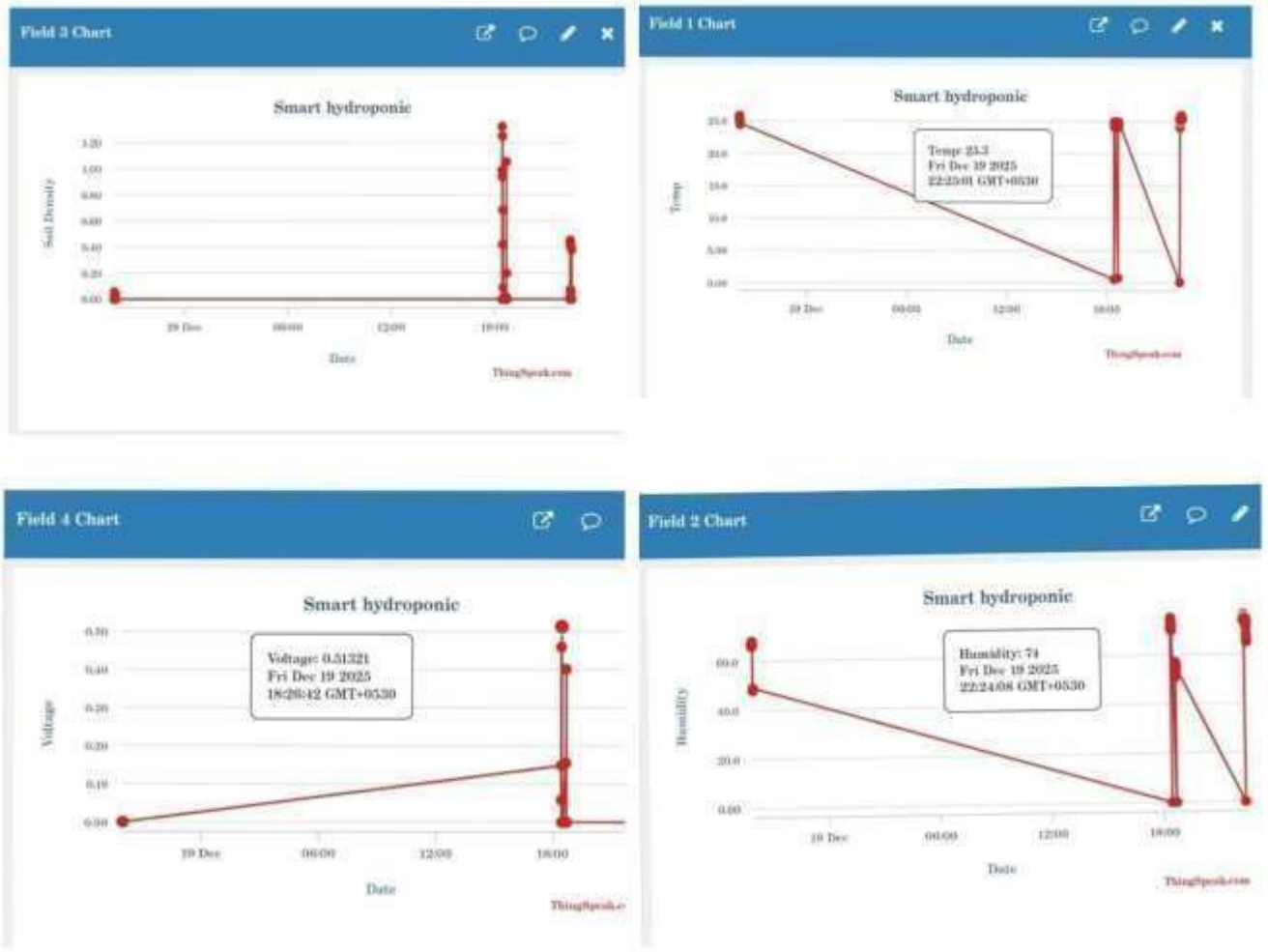


Fig 7.4 ThinkSpeak Data Visualization

7.1 Conclusion

Thus the “greenhouse monitoring system based on the ESP32 microcontroller” has been designed and tested successfully achieves continuous and real-time tracking of environmental factors that influence plant health. The system combines temperature and humidity sensing, soil moisture measurement, local data display, and actuator control into a single integrated platform. Sensor readings are wirelessly transmitted to the ThingSpeak cloud using the ESP32’s inbuilt Wi-Fi module, enabling remote access to live and historical data for effective greenhouse supervision. Local monitoring is supported through an I2C-based LCD, which ensures data visibility even when internet connectivity is unavailable. Then the relay-driven water pump illustrates the system’s capability to support both automated and manual irrigation operations. The overall implementation emphasizes affordability, scalability, and ease of deployment, making it suitable for small greenhouse setups and academic demonstrations. The successful operation of the prototype confirms that IoT-driven agricultural solutions can enhance monitoring accuracy, minimize manual involvement, and promote efficient resource utilization, thereby supporting the development of smarter and more sustainable farming environments.

7.2 Future Scope

The current prototype focuses primarily on monitoring and basic control. With further development, the system can be expanded into a fully automated smart greenhouse platform. The following enhancements can be implemented in future iterations:

1. Integration of Advanced Sensors

Additional sensors can improve environmental accuracy:

- DHT22 for higher precision temperature & humidity
- pH sensor for soil acidity monitoring
- TDS sensor for water purity
- LDR for light intensity measurement
- CO_2 sensor for air quality control

2. Mobile App Integration

- Use Blynk/Firebase/MQTT to build a custom mobile application
- Push notifications for low soil moisture or high temperature
- Real-time dashboard and remote pump control

3. Solar Power Implementation

- Adding a solar panel and battery unit for outdoor or farm deployment
- Reduces power dependency and improves portability

4. Machine Learning

Predictions Using historical

ThingSpeak data to:

- Predict irrigation needs
- Forecast temperature and humidity patterns
- Improve plant growth conditions

5. Automated Ventilation and Lighting

- Adding servos for opening/closing greenhouse vents
- Controlling grow lights using LDR feedback
- Automatic fan activation based on temperature

6. Data Logging and Analytics

- Storing long-term sensor data for trend analysis
- Exporting CSV files from ThingSpeak
- Generating reports for research and farming optimization

REFERENCES

[1] M. A. Zamora-Izquierdo, J. Santa, J. A. Martínez, V. Martínez, and A. F. Skarmeta "Smart Farming IoT Platform Based on Edge and Cloud Computing," Published in International Journal on Computers and Electronics in Agriculture, Vol. 177, issue pp. 105–117, 2020.

[2] Ruiz-Garcia, L. Lunadei, P. Barreiro, and J. I. Robla, "A Review of Wireless Sensor Technologies and Applications in Agriculture," Published in the International Journal on Sensers,Vol. 9, Issue 6, pp. 4728–4750, 2009.

[3] A. Jawad, R. Nordin, S. Gharghan, A. Jawad, and M. Ismail, "Energy-Efficient Wireless Sensor Networks for Precision Agriculture," Published in the International journal on Sensors, Vol. 17, Issue 8, pp. 1781–1795, 2017.

[4] J. Gutierrez, J. F. Villa-Medina, A. Nieto-Garibay, and M. A. Porta-Gandara "Automated Irrigation System Using a Wireless Sensor Network," Published International Journal on IEEE Transactions on Instrumentation and Measurement, Vol. 63, Issue 1, pp. 166–176, 2014.

[5] S. Li, L. Da Xu, and S. Zhao, "The Internet of Things: A Survey," Published in the International Journal on Information Systems Frontiers, Vol.17, Issue 2, pp. 243–259, 2015.

[6] B. George and L. Mathew, "Smart Greenhouse with Real-Time Monitoring Using ESP8266," Published in the International Conference Proceedings Journal on Procedia Computer Science, Vol. 185, pp. 418–424, 2021.

[7] P. K. Pandit, G. M. Jiwane, Z. M. Sheikh, P. J. Wankhede, P. R. Yelekar & M. M. Ramteke, "IoT Based Green House Monitoring and Controlling System," Published in the International Journal of Research Publication and Reviews, Vol. 6, Issue 6, pp. 10250–10253, June 2025.

APPENDIX

```
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27, 16,2);


#include <DHT.h>

#define dhtpin 2

#define dhttype DHT11

DHT dht (dhtpin, dhttype);

//#define gas sensorpin 34

#define gastype mq3


#include "ThingSpeak.h"

#include <WiFi.h>

int valve=12;

int relay=13;


char ssid[] ="";  // your network SSID (name) char

pass[] ="";   // your network password

int keyIndex = 0;            // your network key Index number (needed only for WEP)

WiFiClient  client;

unsigned long myChannelNumber= ;

const char * myWriteAPIKey = " ";


String myStatus = "";


//float t, h, val, volt, info, solar;

float t, h, val, d, info, v;

void setup()
```

```
{
  lcd.init();
  lcd.backlight();
  lcd.begin(16,2);


  dht.begin();
  ThingSpeak.begin(client);
  Serial.begin(115200);
  pinMode(valve,OUTPUT);
  pinMode(relay,OUTPUT);
  delay(100);
 }


void loop()
{
  if(WiFi.status() != WL_CONNECTED)
  {
    Serial.print("Attempting to connect to SSID: ");
    Serial.println(ssid);
    while(WiFi.status() != WL_CONNECTED)
    {
      WiFi.begin(ssid, pass);
      Serial.print(".");
      delay(5000);
    }
    Serial.println("\nConnected.");
  }
```

```
 t=dht.readTemperature();
    h=dht.readHumidity();


    Serial.print("Temperature: ");
    Serial.print(t);
    Serial.print("\xC2\xB0");
    Serial.print("C");
    Serial.print("\t\t");
    Serial.print("Humidity: ");
    Serial.print(h);
    Serial.println("%");


/*if(mqtt.connected());
//if(gas sensor.connected())
int Gas_value = analogRead(34);
Serial.println(Gas_value);
Serial.print ("...") ;
//if (Gas_data.publish(Gas_value))
//void GASLevel() {
  int value = analogRead(sensor);
 value = map(value, 0, 4095, 0, 100);
 Serial.print("bad: ");
    Serial.print(PPM);
    Serial.println();
    delay(500);
```

```
  val = analogRead(35);
   Serial.println(info);
   d = val*34.1/4096.0;
  Serial.print("bad smile: ");
  Serial.print(d);
  Serial.println("D");

 info = analogRead(34);
  Serial.println(info);
  v = info*4.9/4096.0;
 //v = map(value, 0, 4095, 0, 100);
  Serial.print("full: ");
  Serial.print(v);
  Serial.println("v");
  Serial.println();
  delay(500);

  lcd.setCursor(0,0);
  lcd.print("T: ");
  lcd.print(t);
  lcd.setCursor(9,0);
  lcd.print("H: ");
  lcd.print(h);

  lcd.setCursor(0,1);
  lcd.print("D: ");
  lcd.print(d);
```

```
  lcd.setCursor(9
  ,1);
  lcd.print("v: ");
  lcd.print(v);
  if(t>31)
 {
  digitalWrite(valve,1);
  //delay(50);
 //digitalWrite(valve,0);
lcd.setCursor(0,1);
lcd.print("valve on");
lcd.print("     ");
 }
if(t<31 )
 {
 digitalWrite(valve,0);
  //delay(1000);
 // digitalWrite(relay,0);
  // digitalWrite(valve,0);
 //  lcd.setCursor(0,1);
 lcd.print("valve off");
 //  lcd.print("       ");
 }

  ThingSpeak.setField(1,  t);
  ThingSpeak.setField(2,  h);
  ThingSpeak.setField(3,  d);
   ThingSpeak.setField(4, v);
```

```
  // ThingSpeak.setField(5, i);
 // write to the ThingSpeak channel

   int x = ThingSpeak.writeFields(myChannelNumber, myWriteAPIKey);

   if(x == 200)

   {

     Serial.println("Channel update successful.");

     Serial.println();

   }

   else

   {

     Serial.println("Problem updating channel. HTTP error code " + String(x));

     Serial.println();

   }


   // change the values

   ThingSpeak.writeFields(myChannelNumber, myWriteAPIKey);

   delay(15000); // Wait 15 seconds to update the channel again

}
```